

WEEK 1

MICROCONTROLLER AND PROCESSOR FUNDAMENTALS

1.1 INTRODUCTION

The first week of the internship focused on understanding the fundamentals of embedded systems, microcontrollers and processors. The training covered the basic architecture of an embedded system and the interaction between hardware and software.

The major topics studied during this week included microcontroller architecture, processor fundamentals, memory organization, Embedded C programming, GPIO, digital interfacing, interrupts, timers, counters, clock and reset operations. Practical activities were also carried out to understand how software instructions control electronic hardware.

1.2 OBJECTIVES

The main objective of Week 1 was to understand the working principles of embedded systems and the role of microcontrollers and processors in electronic applications. The training also focused on memory organization, Embedded C programming and basic hardware interfacing.

The practical objective was to understand how a controller receives an input, processes it according to a program and produces the required output.

1.3 EMBEDDED SYSTEM FUNDAMENTALS

An embedded system is an electronic system designed to perform a specific function or a predefined set of functions. It combines hardware and software to perform the required operation.

The hardware generally consists of a microcontroller or processor, memory, input and output interfaces, communication peripherals and a power supply. The software contains the instructions required to control these hardware components.

For example, an automatic washing machine uses an embedded controller to monitor switches and sensors and control the motor, water valve and other components. Similarly, an IoT device can collect information from sensors, process the information and transmit it to another device or cloud platform.

A basic embedded system follows the sequence:

Input → Processing → Output

where sensors and switches provide inputs, the microcontroller processes the information and actuators or communication interfaces provide the required output.

BLOCK DIAGRAM OF A MICROCONTROLLER SYSTEM

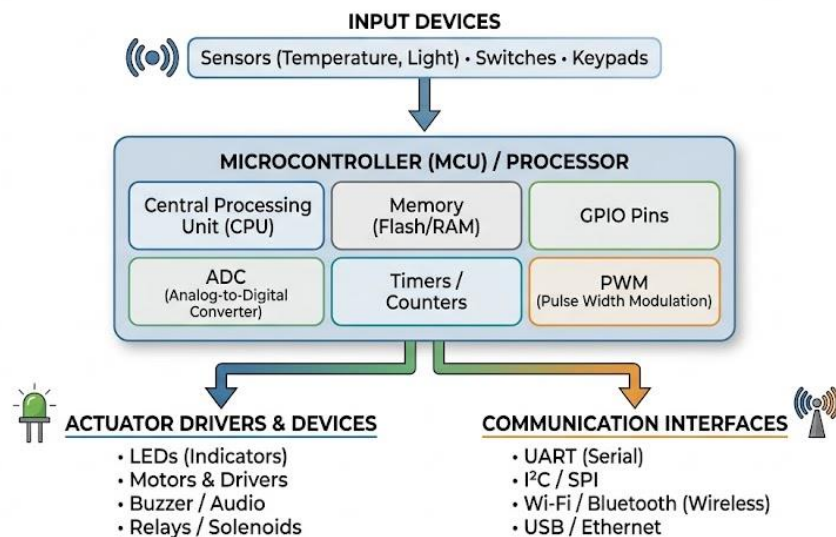


Figure 1.1 – Basic Embedded System Architecture

1.4 MICROCONTROLLER AND ITS ARCHITECTURE

A microcontroller is a compact integrated circuit that contains a processor, memory and peripheral interfaces on a single chip. It is widely used in embedded systems because it can directly interface with electronic components.

A typical microcontroller contains a CPU, Flash memory, RAM, GPIO ports, timers, counters, ADC, PWM and communication peripherals such as UART, I²C and SPI.

The CPU executes program instructions, while memory stores the program and temporary data. GPIO provides digital input and output connections. Timers and counters provide timing and counting functions, while ADC allows analog signals to be converted into digital values.

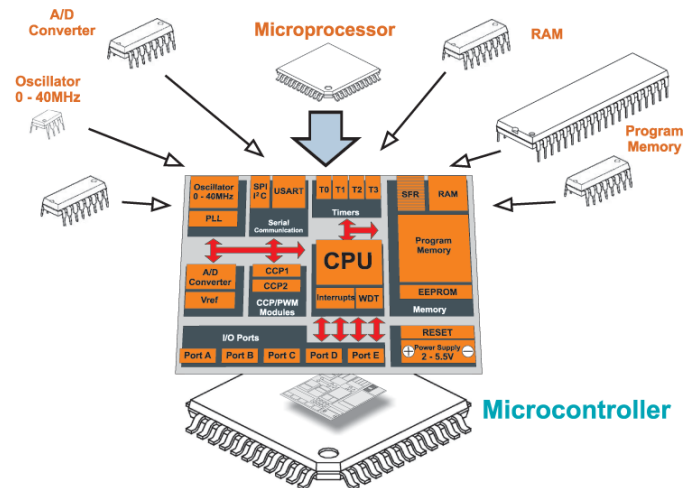


Figure 1.2 – Basic Microcontroller Architecture

The integration of these components makes a microcontroller suitable for compact and dedicated control applications.

1.5 PROCESSOR AND CPU FUNDAMENTALS

The Central Processing Unit is the main processing section of a microcontroller or processor. It executes instructions and controls the operation of the system.

The CPU mainly consists of the Arithmetic Logic Unit, control unit and registers. The ALU performs arithmetic and logical operations, while registers provide fast temporary storage for data and addresses.

The Program Counter stores the address of the next instruction to be executed. The Stack Pointer keeps track of the stack used during function calls and interrupt handling.

The basic instruction execution process is:

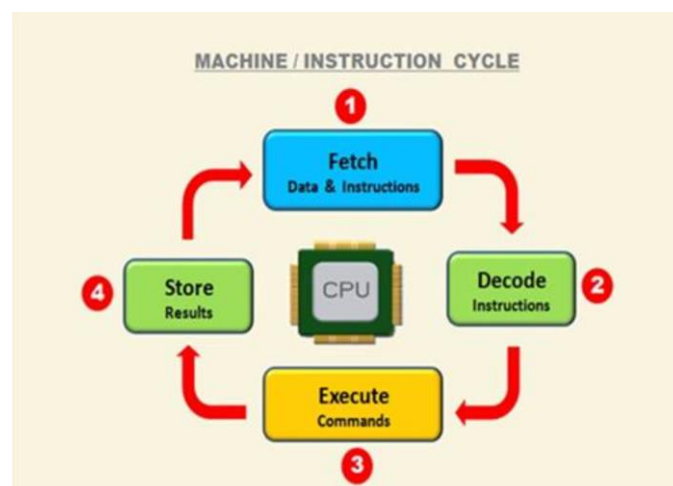


Figure 1.3 – Basic Processor Instruction Cycle

1.6 MEMORY ORGANIZATION

Memory is required to store program instructions, variables and other information used by an embedded system.

RAM is volatile memory used for temporary data during program execution. Its contents are normally lost when the power supply is removed. Flash memory is non-volatile memory commonly used to store the firmware or program of a microcontroller. The stored program remains available even after power is removed.

EEPROM is another type of non-volatile memory that can be used to store small amounts of permanent information such as configuration parameters and calibration values.

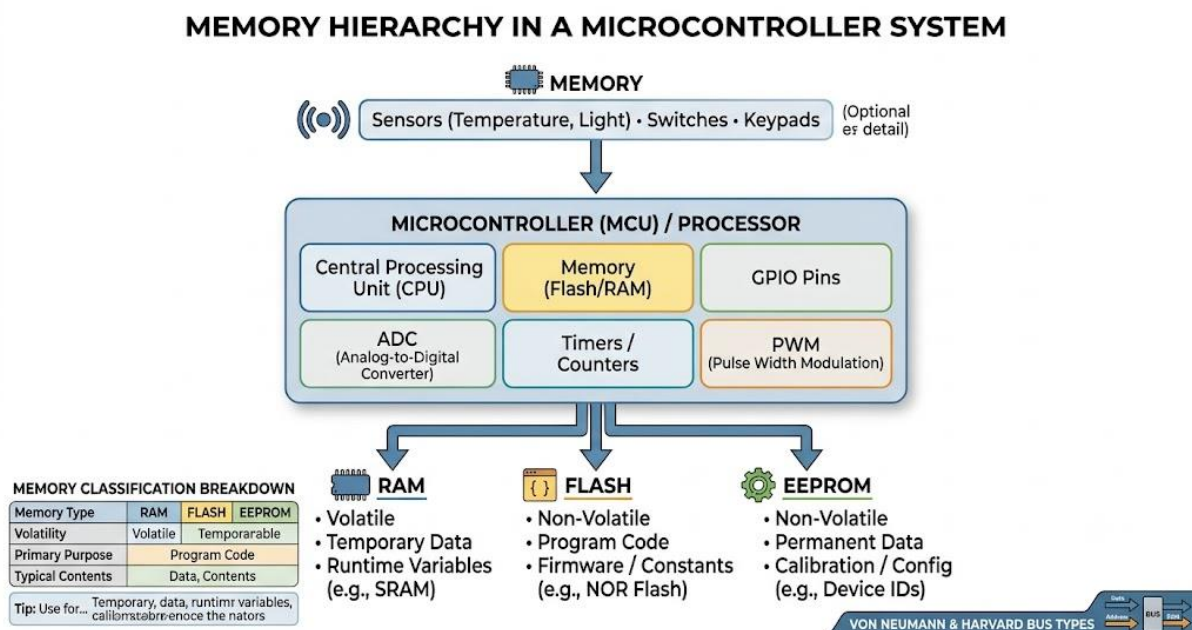


Figure 1.4 – Memory Organization

The memory requirements of an embedded system depend on the program size, amount of data and complexity of the application.

1.7 MICROCONTROLLER AND MICROPROCESSOR

A microcontroller integrates the processor, memory and peripherals into a single chip, whereas a microprocessor primarily provides the processing unit and generally requires external memory and peripheral components.

Microcontrollers are commonly used for dedicated control applications because they require fewer external components and generally consume less power. Microprocessors are more suitable for applications requiring higher processing capability and complex operating systems.

Feature	Microcontroller	Microprocessor
CPU	Integrated	Integrated
RAM	Usually integrated	Usually external
Program memory	Usually integrated	Usually external
GPIO	Usually integrated	Often external
Power consumption	Generally lower	Generally higher
Application	Dedicated control	Complex computing
Examples	AVR, STM32, ESP32	ARM Cortex-A, Intel processors

Table 1.1 – Comparison of Microcontroller and Microprocessor

This comparison helped in understanding the selection of a suitable processing device for an embedded application.

1.8 EMBEDDED C PROGRAMMING

Embedded C was introduced as the primary programming method for controlling microcontroller hardware. It is based on the C programming language but is used with hardware-specific registers, peripherals and memory locations.

The training covered variables, data types, operators, conditional statements, loops, functions, arrays and pointers. Bitwise operations were also introduced because they are frequently used to configure and control individual bits in hardware registers.

A typical embedded program follows:

Hardware Initialization → Input Reading → Data Processing → Output Control

For example, a GPIO pin can be configured as an output and controlled through an Embedded C program to switch an LED ON or OFF.

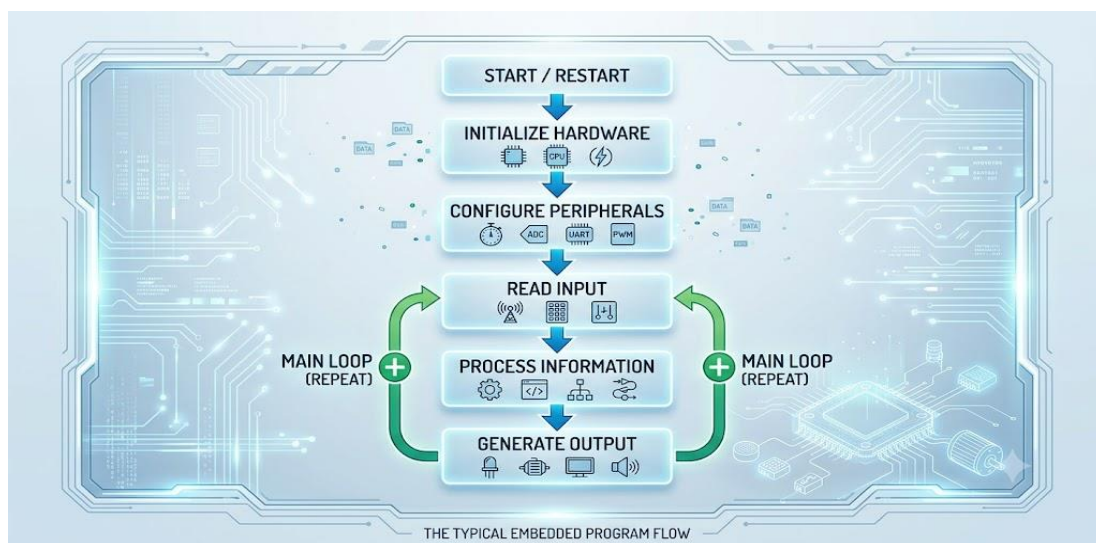


Figure 1.5 – Embedded C and Hardware Interaction

1.9 GPIO AND DIGITAL INTERFACING

GPIO stands for General Purpose Input and Output. GPIO pins allow a microcontroller to communicate directly with digital electronic components.

A GPIO configured as an input can read a push button or digital sensor. A GPIO configured as an output can control an LED, buzzer, relay or other digital device.

Pull-up and pull-down resistors are used to maintain a defined logic level when an input is not actively driven. They help prevent an input from remaining in an undefined or floating state.

1.9.1 LED Interfacing

LED interfacing was studied as a simple digital output application. The LED is connected to a GPIO pin through a suitable current-limiting resistor. The microcontroller controls the GPIO state through the program to switch the LED ON or OFF.

1.9.2 Push Button Interfacing

A push button was used as a digital input. The microcontroller reads the voltage level of the GPIO pin and determines whether the button is pressed or released.

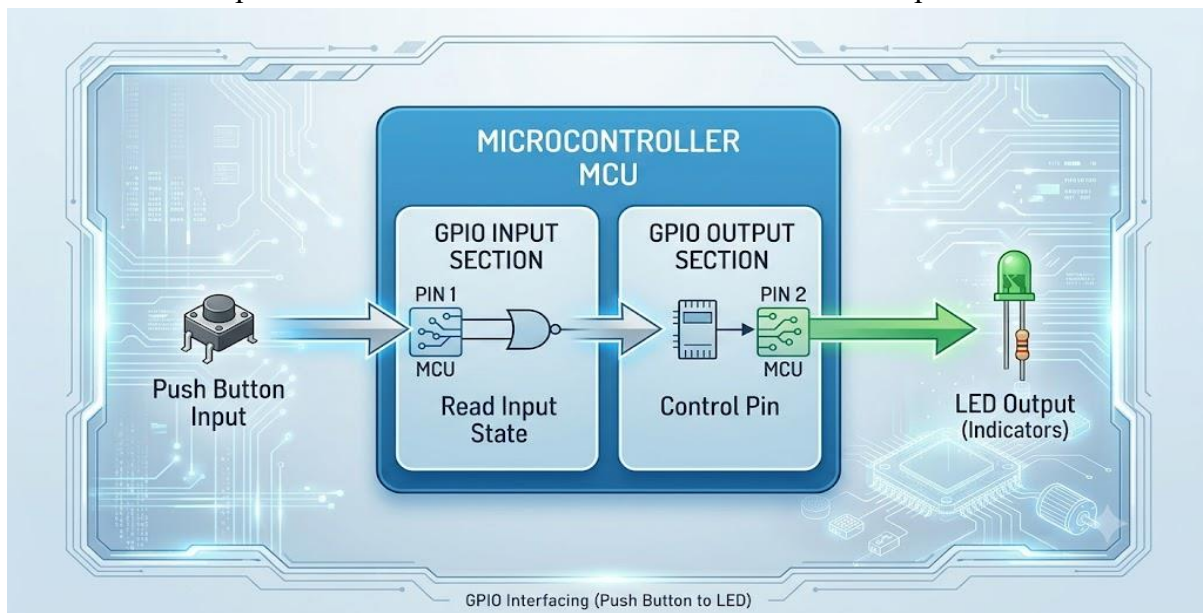


Figure 1.6 – Push Button and LED Interfacing

These experiments demonstrated the relationship between software instructions and physical hardware.

1.10 INTERRUPTS

An interrupt is a mechanism that allows the processor to respond to an event without continuously checking the input condition.

During normal operation, the processor executes the main program. When an interrupt occurs, the processor temporarily pauses the current operation, executes the corresponding Interrupt Service Routine and then returns to the main program.

Interrupts can be generated by external GPIO events, timers, communication peripherals and other hardware events.

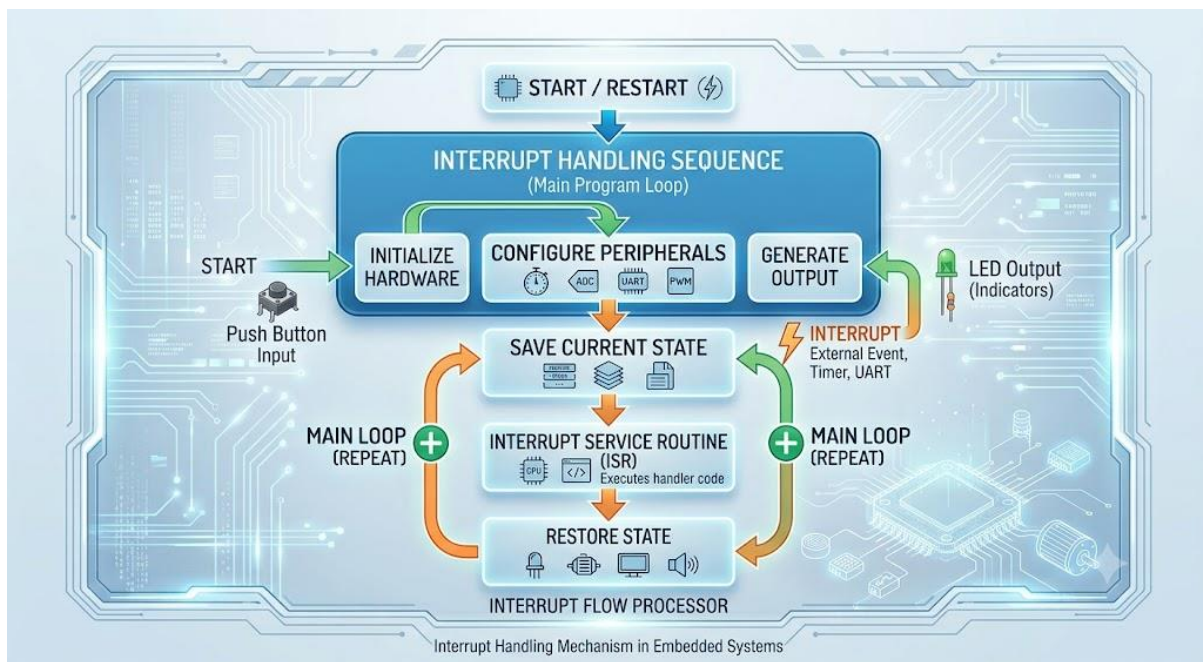


Figure 1.7 – Interrupt Operation

Interrupts are useful when the controller must respond quickly to an event while continuing other processing tasks.

1.11 POLLING AND INTERRUPTS

Polling and interrupts are two common methods of detecting events.

In polling, the processor repeatedly checks the status of an input or peripheral. Although this method is simple, it can consume processor time.

In an interrupt-based system, the processor continues executing the main program until the required event occurs. The hardware then generates an interrupt and the processor executes the corresponding interrupt routine.

Therefore, interrupts are generally preferred when fast response to an event is required.

1.12 TIMERS, COUNTERS, CLOCK AND RESET

Timers and counters are important peripherals used for time measurement and event counting.

A timer uses the controller clock to generate time intervals or periodic events. Timers can be used for delays, periodic operations, PWM generation and time measurement. A counter is used to count external pulses or events, such as the number of rotations or objects detected by a sensor.

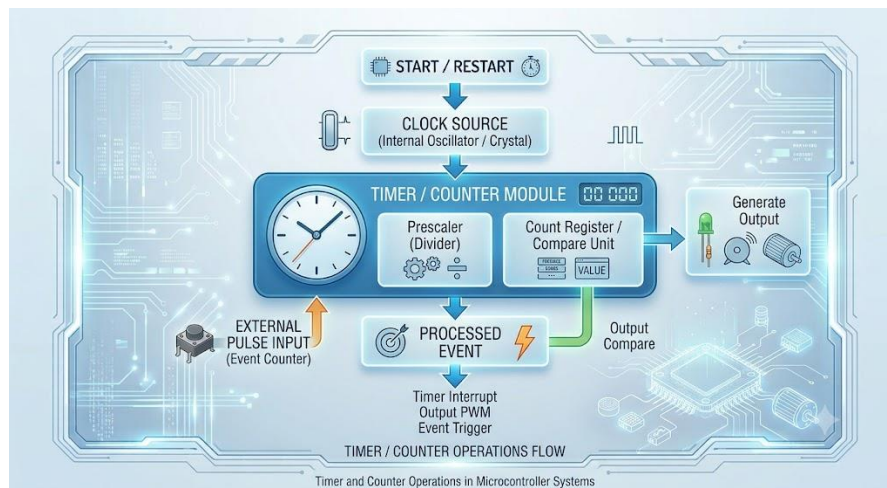


Figure 1.8– Timer and Counter Concept

The clock provides the timing reference required by the processor and peripherals. Clock frequency influences the operating speed of the controller and timing of peripheral operations.

The reset system initializes the controller to a known state. Reset may occur during power-up, through an external reset signal or due to certain system conditions.

1.13 WEEK 1 LEARNING OUTCOME

At the end of the first week, the fundamental operation of embedded systems and microcontrollers was understood. The training provided knowledge of processor operation, memory organization, Embedded C, GPIO, digital interfacing, interrupts, timers, counters, clock and reset.

The practical activities helped connect theoretical concepts with actual hardware operation and prepared the foundation for working with sensors and communication interfaces.

WEEK 2

EMBEDDED SYSTEM DEVELOPMENT AND COMMUNICATION INTERFACES

2.1 INTRODUCTION

The second week of the internship focused on extending the microcontroller fundamentals learned during Week 1 into practical embedded system development.

The major topics covered were analog and digital signals, ADC, PWM and serial communication protocols. UART, I²C and SPI were studied because these interfaces are commonly used to connect microcontrollers with sensors, displays, memory devices and other peripheral modules.

The week also included ARM processor fundamentals, pointers, bit manipulation, sensor data acquisition and basic debugging techniques.

2.2 OBJECTIVES

The objective of Week 2 was to understand how microcontrollers process analog signals and communicate with external peripherals. The training also focused on commonly used communication protocols and basic techniques for testing and debugging embedded systems.

2.3 ANALOG AND DIGITAL SIGNALS

Electronic signals can be broadly classified as analog and digital.

An analog signal varies continuously with time and can have different voltage levels within a specified range. Temperature, light intensity, pressure and sound can be represented by analog signals.

A digital signal uses discrete logic levels, commonly represented by logic HIGH and LOW or binary 1 and 0.

Since microcontrollers process digital information internally, analog signals must be converted into digital values before they can be processed.

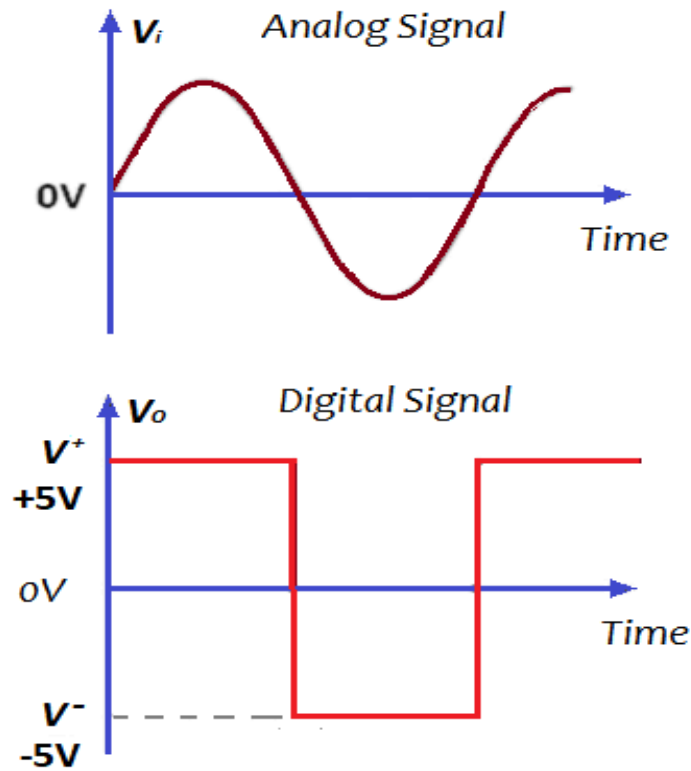


Figure 2.1 – Analog and Digital Signals

2.4 ADC – ANALOG TO DIGITAL CONVERTER

An Analog-to-Digital Converter converts an analog voltage into a digital numerical value that can be processed by a microcontroller.

For an N-bit ADC, the number of available digital levels is:

$$\text{Number of levels} = 2^N$$

Therefore, an 8-bit ADC provides 256 levels from 0 to 255, while a 10-bit ADC provides 1024 levels from 0 to 1023.

The approximate digital output can be represented as:

$$\text{Digital Value} = (V_{in} / V_{ref}) \times (2^N - 1)$$

where V_{in} is the input voltage, V_{ref} is the reference voltage and N is the ADC resolution.

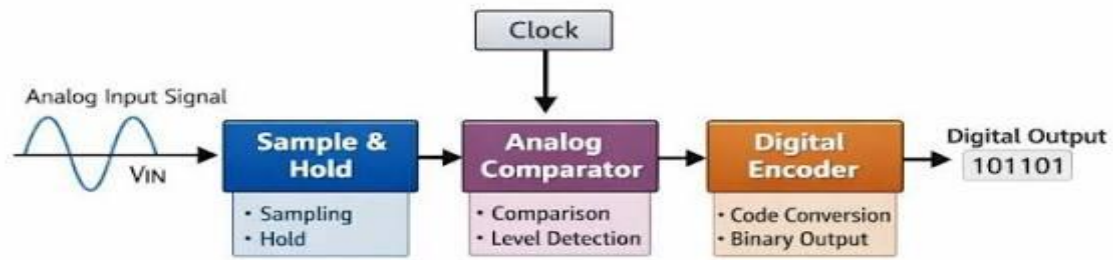


Figure 2.2 – ADC Working

ADC is important in sensor-based systems because many sensors produce analog electrical signals corresponding to physical parameters.

2.5 PWM – PULSE WIDTH MODULATION

Pulse Width Modulation is a technique used to control the average power delivered to a device by changing the ON time of a digital signal.

The duty cycle is given by:

$$\text{Duty Cycle} = (\text{TON} / T) \times 100$$

where TON is the ON time and T is the total signal period.

For example, a 50% duty cycle means that the signal remains HIGH for half of each cycle.

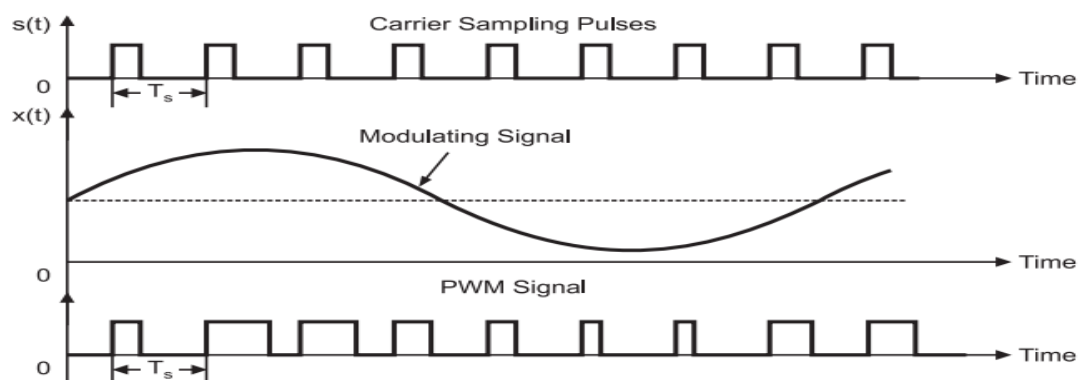


Figure 2.3 – PWM Waveform

PWM is commonly used for controlling LED brightness, DC motor speed and servo motor position.

2.6 UART COMMUNICATION

UART stands for Universal Asynchronous Receiver/Transmitter. It is a serial communication method commonly used for communication between a microcontroller and another device.

The main communication signals are TX and RX. TX is used for transmitting data and RX is used for receiving data. A common ground connection is also required.

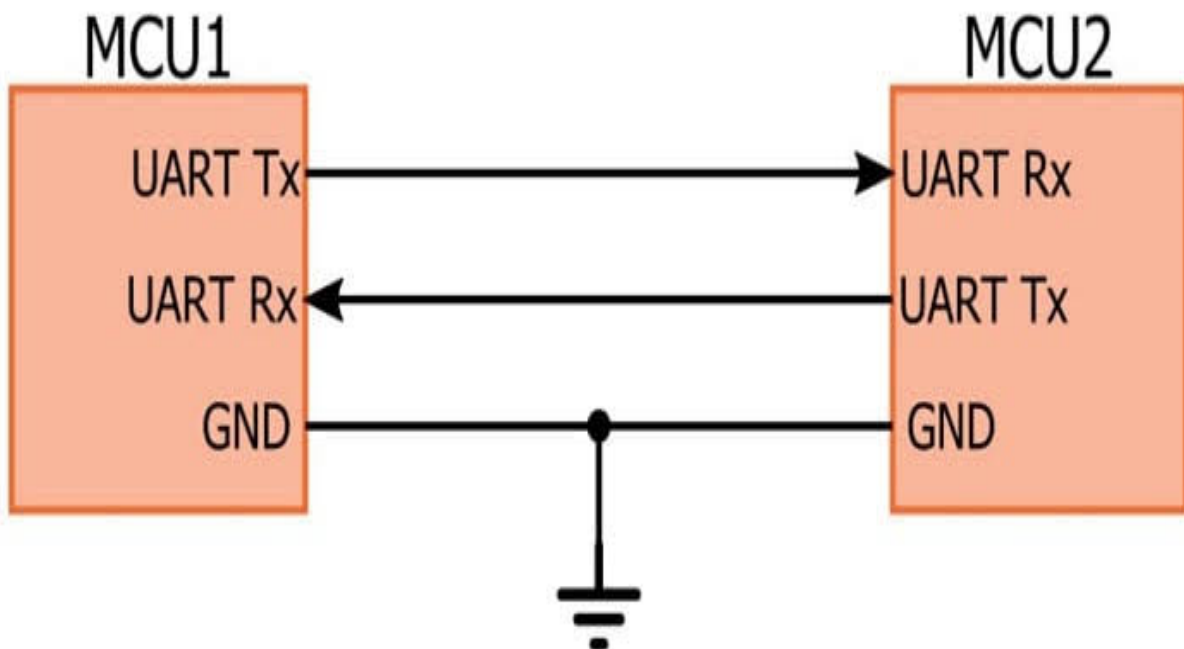


Figure 2.4 – UART Communication

UART communication requires compatible settings between the communicating devices. Important parameters include baud rate, number of data bits, parity and stop bits.

UART is also widely used for debugging because sensor readings, system status and error messages can be displayed on a computer through a serial monitor.

2.7 I²C COMMUNICATION

I²C, or Inter-Integrated Circuit, is a synchronous serial communication protocol that uses two main lines: SDA and SCL.

SDA carries data, while SCL carries the clock signal. Multiple devices can share the same bus, with each device identified by its address.

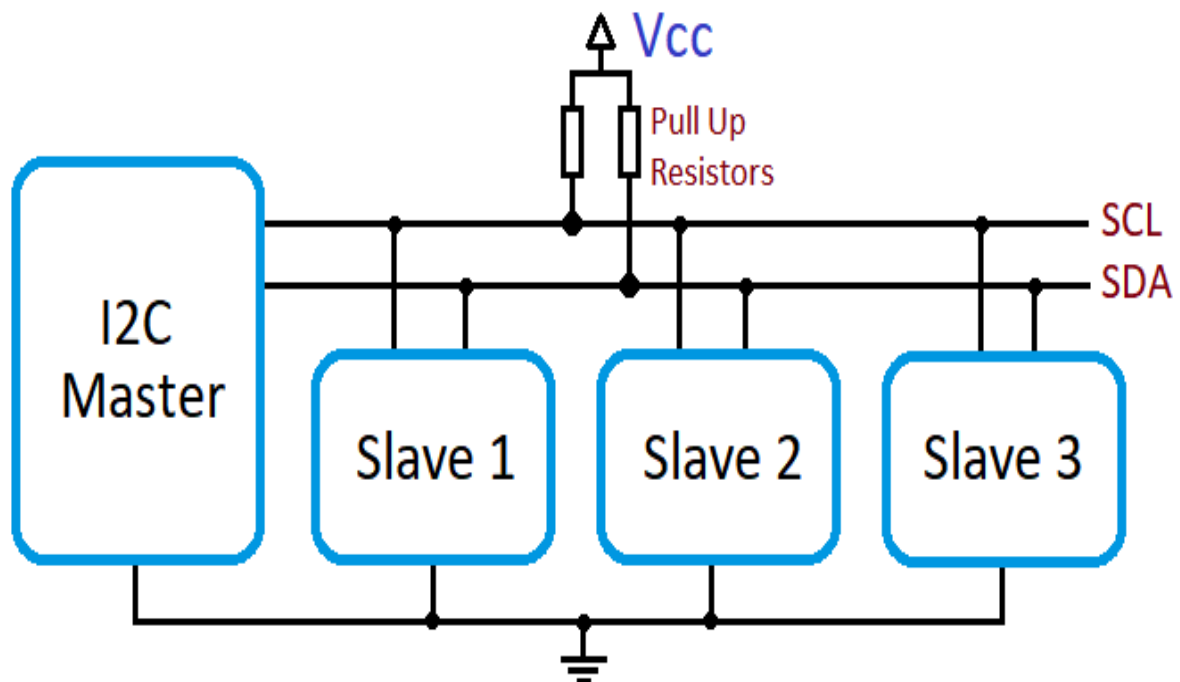


Figure 2.5 – I²C Communication

I²C is commonly used for connecting sensors, OLED displays, RTC modules and EEPROM devices to microcontrollers.

2.8 SPI COMMUNICATION

SPI stands for Serial Peripheral Interface. It is a synchronous serial communication protocol used to connect a controller with one or more peripheral devices.

The main signals are MOSI, MISO, SCLK and CS/SS. MOSI transfers data from the master to the slave, while MISO transfers data from the slave to the master. SCLK provides the clock and CS selects the required peripheral.

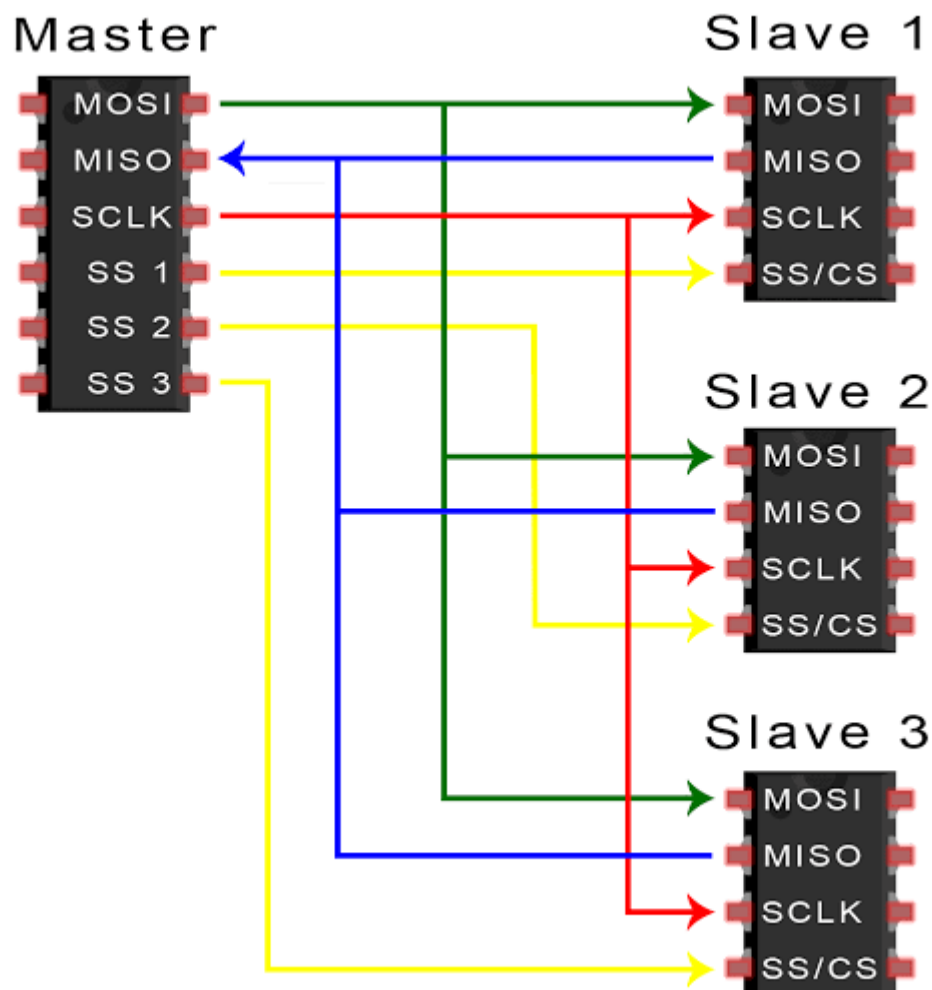


Figure 2.6 – SPI Communication

SPI is suitable for applications requiring relatively fast communication with peripheral devices.

2.9 COMPARISON OF UART, I²C AND SPI

Parameter	UART	I ² C	SPI
Type	Asynchronous	Synchronous	Synchronous
Main signals	TX, RX	SDA, SCL	MOSI, MISO, SCLK, CS
Clock	Not required	Required	Required
Device selection	Not applicable	Addressing	Chip Select
Common applications	Debugging, serial communication	Sensors, displays, RTC	Fast peripherals

Table 2.1 – Comparison of UART, I²C and SPI

The appropriate communication protocol is selected based on speed, number of devices, wiring requirements and application needs.

2.10 ARM PROCESSOR FUNDAMENTALS

ARM-based processors are widely used in embedded and IoT applications because of their performance and power efficiency.

The training introduced basic ARM concepts such as registers, memory addressing, instruction execution, interrupts and peripheral control.

ARM-based devices are available in different classes. ARM-based microcontrollers are commonly used for control applications, while higher-performance ARM application processors can support complex operating systems and advanced applications.

2.11 POINTERS AND BIT MANIPULATION IN EMBEDDED C

Pointers are an important part of Embedded C because they allow programs to work directly with memory addresses.

For example:

```
int value = 10;  
int *ptr = &value;
```

Here, value stores the data and ptr stores the address of value.

Pointers are useful for handling arrays, passing data to functions, accessing memory locations and working with hardware registers.

Bit manipulation is also important because microcontroller registers contain several individual control and status bits. Common bitwise operators include AND (&), OR (|), XOR(^), NOT (~), left shift (<<) and right shift (>>).

For example:

```
PORT |= (1 << 2);
```

This operation sets bit 2 of the specified register.

These techniques allow individual hardware functions to be controlled efficiently.

2.12 SENSOR DATA ACQUISITION

Sensor data acquisition involves detecting a physical parameter and converting it into useful electrical and digital information.

A sensor may measure parameters such as temperature, light intensity, pressure or humidity. Depending on the sensor, the output may be analog or digital. An analog output can be connected to an ADC, while a digital sensor can communicate directly through a suitable digital interface.

The general data acquisition process is:

**Physical Parameter → Sensor → Electrical Signal → ADC/Interface →
Microcontroller → Data Processing**

The processed sensor information can later be displayed locally or transmitted to another system using a communication interface.

2.13 SERIAL MONITORING AND DEBUGGING

Serial communication was also used as a basic debugging technique. A microcontroller can transmit sensor values, program status and error messages to a computer through UART.

For example, continuously displaying sensor readings on a serial monitor makes it possible to verify whether the sensor and program are operating correctly.

Debugging therefore involves checking both hardware and software. Power supply, wiring, pin configuration, program logic and communication settings must be verified systematically.

2.14 TROUBLESHOOTING EMBEDDED SYSTEMS

Troubleshooting is an essential part of embedded system development because problems may occur in hardware connections, software configuration, power supply or communication.

A systematic troubleshooting approach was studied:

Power Supply Check → Hardware Connection Check → Pin Configuration Check → Program Check → Communication Check → Individual Component Test → Complete System Test

This approach helps identify the source of a problem without changing multiple parameters at the same time.

2.15 PRACTICAL ACTIVITY – COMMUNICATION INTERFACES

UART, I²C and SPI were studied as practical communication interfaces for embedded systems.

UART was considered for point-to-point communication and serial debugging. I²C was studied for connecting multiple peripherals using a shared two-wire bus. SPI was studied for communication with peripherals where higher data transfer speed is required.

These interfaces are commonly used when developing sensor and IoT systems because they provide a method for exchanging information between the microcontroller and external devices.

2.16 WEEK 2 LEARNING OUTCOME

At the end of Week 2, the operation of analog and digital signals, ADC, PWM and serial communication interfaces was understood. Practical knowledge of UART, I²C and SPI were obtained along with basic ARM processor concepts, pointers and bit manipulation.

The sensor interfacing and debugging activities helped in understanding how an embedded system can acquire information, process it and communicate the result to another device.

WEEK 3

MICROCONTROLLERS, SENSORS, ACTUATORS AND IoT CLOUD COMMUNICATION

3.1 INTRODUCTION

The third week of the internship focused on the fundamental concepts and practical applications of microcontrollers, sensors, actuators and IoT communication technologies. The training provided an understanding of how electronic devices collect information, process signals and communicate data through wired and wireless methods.

The major topics covered during the week included Arduino Uno, ESP8266, different types of sensors, actuators, OLED displays, Wi-Fi connectivity and cloud platforms. These topics provided practical knowledge of embedded systems and IoT technologies.

3.2 OBJECTIVES

The main objective of the training was to understand the basic operation and applications of microcontrollers, sensors and actuators in embedded systems.

The training also focused on understanding sensor interfacing, output device control, Wi-Fi connectivity and transmission of data to an IoT cloud platform.

3.3 MICROCONTROLLERS

3.3.1 Arduino Uno

Arduino Uno is a microcontroller development board based on the ATmega328P microcontroller. It is widely used for embedded systems, automation, robotics and electronic prototyping.

The board provides digital and analog input/output pins that can be used to interface sensors and electronic devices. It can receive signals from external devices, process the information according to programmed instructions and generate appropriate output signals.

Arduino Uno can be programmed using the Arduino IDE with C/C++-based programming. It provides a simple platform for learning microcontroller programming, input/output control and sensor interfacing.



Figure 3.1 – Arduino Uno

3.3.2 ESP8266

ESP8266 is a low-cost microcontroller module with built-in Wi-Fi connectivity. It is commonly used in Internet of Things applications because it can communicate with wireless networks and Internet-based services.

The ESP8266 can receive data from sensors or other controllers, process the information and transmit it wirelessly. Its integrated Wi-Fi capability makes it suitable for applications involving remote monitoring and cloud communication.

The module can be programmed using the Arduino IDE, which provides a simple development environment for creating Wi-Fi-enabled embedded applications.



Figure 3.2 – ESP8266

3.4 SENSORS

3.4.1 Introduction To Sensors

A sensor is an electronic device that detects a physical quantity or environmental condition and converts it into an electrical signal.

Sensors can be classified as analog or digital depending on the type of output they provide. Analog sensors generate continuously varying electrical signals, while digital sensors generally provide discrete or digitally formatted information.

Sensors are widely used in embedded systems to measure parameters such as temperature, humidity, pressure, light, moisture, distance and motion.

3.4.2 DHT11 Temperature And Humidity Sensor

The DHT11 is a digital sensor used to measure temperature and relative humidity. It provides the measured values in digital form to a connected microcontroller.

Temperature is generally expressed in degrees Celsius, while relative humidity is expressed as a percentage. The DHT11 is commonly used in basic environmental monitoring and embedded applications.

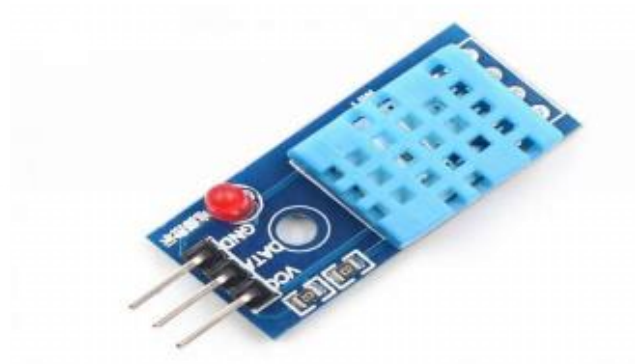


Figure 3.3 – DHT11 Temperature and Humidity Sensor

3.4.3 Soil Moisture Sensor

A soil moisture sensor is used to determine the amount of moisture present in soil. The sensor produces an electrical output that changes according to the moisture condition.

The output can be read by a microcontroller and processed to determine whether the soil is relatively dry or moist. Soil moisture sensors are commonly used in agricultural automation and irrigation applications.

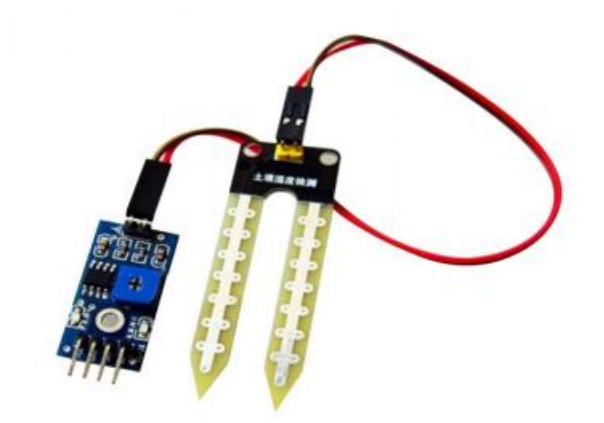


Figure 3.4 – Soil Moisture Sensor

3.4.4 Water Level Sensor

A water level sensor is used to detect or measure the level of water in a container or tank. It produces an electrical signal corresponding to the detected water level.

The output of the sensor can be connected to a microcontroller for measurement, processing and indication. Water level sensors are commonly used in liquid-level monitoring and control applications.



Figure 3.5 – Water Level Sensor

3.4.5 Rain Sensor

A rain sensor is used to detect the presence of water droplets or rainfall. It generally consists of a sensing plate that changes its electrical characteristics when water comes into contact with its surface.

The sensor can provide digital or analog output depending on the module and configuration. It is commonly used in weather monitoring and automatic control applications.

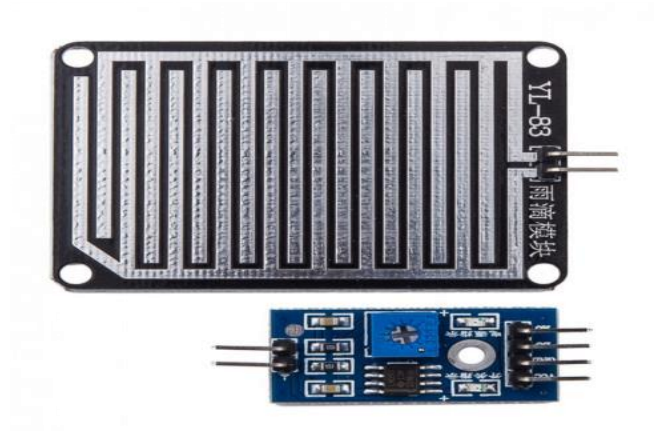


Figure 3.6 – Rain Sensor

3.4.6 Ldr Sensor

LDR stands for Light Dependent Resistor. It is a light-sensitive resistor whose resistance changes according to the intensity of light falling on its surface.

The resistance of an LDR generally decreases when the light intensity increases and increases when the light intensity decreases.

It can be used with an analog circuit to measure changes in surrounding light conditions. LDRs are commonly used in automatic lighting, light detection and brightness monitoring applications.

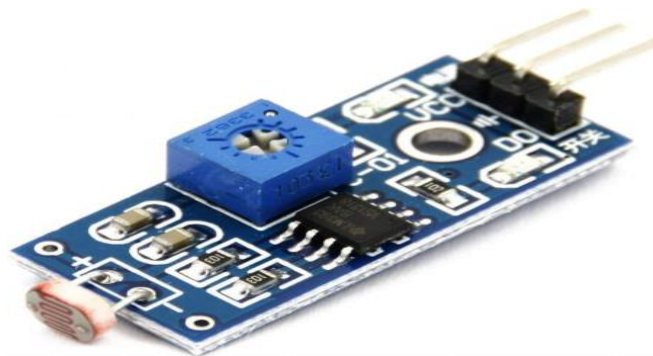


Figure 3.7 – LDR Sensor

3.4.7 Turbidity Sensor

A turbidity sensor is used to measure the cloudiness or suspended particles present in water. It operates by detecting the amount of light scattered or absorbed by particles in the

water. The sensor generally provides an analog output corresponding to the turbidity level. A higher concentration of suspended particles produces a higher turbidity level. Turbidity sensors are commonly used in water-quality monitoring applications to identify changes in water clarity.

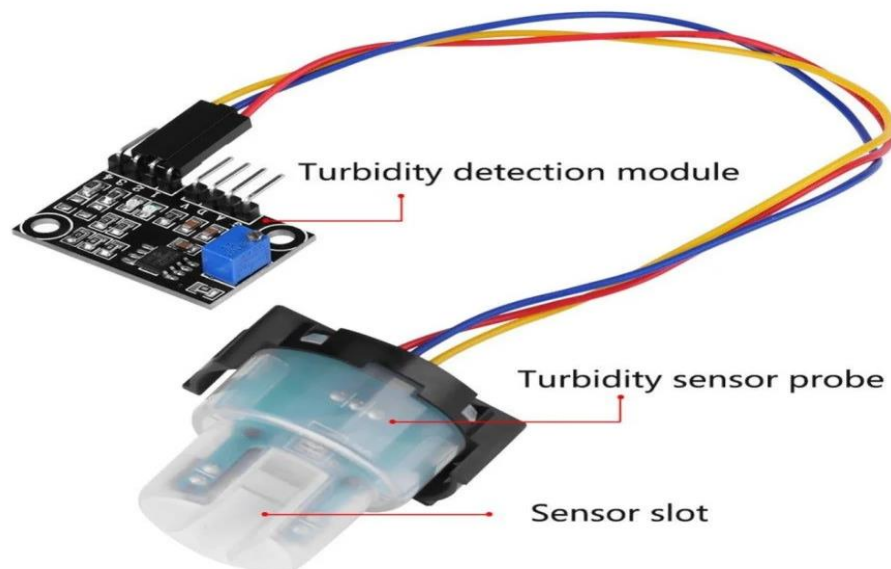


Figure 3.8 – Turbidity Sensor

3.5 ACTUATORS

3.5.1 Introduction To Actuators

An actuator is a device that converts an electrical control signal into a physical action. Actuators are generally controlled by a microcontroller according to the input received from sensors.

Common actuators include motors, pumps, solenoid devices and relays. Output devices such as LEDs, buzzers and displays are also commonly used to provide visual, audible or information-based output.

3.5.2 Relay Module

A relay is an electrically operated switching device used to control electrical loads using a control signal from a microcontroller.

A relay allows a low-power control circuit to switch devices that require higher voltage or current. Relays are commonly used for controlling motors, pumps, lamps and other electrical equipment.

(1 Channel Relay Module) 5V DC 10A Expansion Board



Figure 3.9 – Relay Module

3.5.3 Water Pump

A water pump is an electromechanical device used to move water from one location to another. It converts electrical energy into mechanical energy to create fluid movement.

A microcontroller generally cannot drive a high-power pump directly. Therefore, a relay, transistor driver or motor driver circuit is commonly used between the controller and the pump.



Figure 3.10 – Water Pump

3.5.4 OLED Display

An OLED display is an electronic output device used to display text, numbers and graphical information. OLED stands for Organic Light-Emitting Diode.

A commonly used OLED module is based on the SSD1306 controller and has a resolution of 128×64 pixels. It can communicate with a microcontroller using interfaces such as I²C.

OLED displays are useful for displaying sensor readings, system status and other information in embedded applications.



Figure 3.11 – 128×64 OLED Display

3.5.5 LED and Buzzer

An LED, or Light Emitting Diode, is a semiconductor device that produces light when current flows through it. LEDs are commonly used as visual indicators in electronic systems.

A buzzer is an electronic output device that produces an audible sound when an electrical signal is applied. Buzzers can be used to provide sound indications for different system conditions.



Figure 3.12 – LED and Buzzer

3.6 IoT AND CLOUD COMMUNICATION

The Internet of Things (IoT) refers to the connection of physical devices to communication networks so that they can collect, exchange and process data.

IoT systems generally consist of sensors for collecting information, a controller for processing the data, a communication module for transferring information and a cloud platform for storage and visualization.

Cloud platforms provide online services for storing, processing and displaying data received from connected devices. This enables information collected by embedded devices to be accessed remotely through the Internet.

3.6.1 Wi-Fi Communication Using ESP8266

Wi-Fi is a wireless communication technology used to connect electronic devices to a local network and the Internet.

The ESP8266 contains an integrated Wi-Fi module that allows it to connect to wireless networks. It can send and receive data through the Internet and communicate with web servers and cloud platforms.

Wi-Fi connectivity is an important feature in IoT systems because it enables embedded devices to communicate with remote services.

3.6.2 ThingSpeak Cloud Platform

ThingSpeak is an IoT cloud platform used to collect, store and visualize data received from connected devices.

Data can be sent to different fields within a ThingSpeak channel. Each field can represent a particular parameter, and the received values can be displayed using numerical information or graphs.

ThingSpeak provides a simple platform for understanding IoT data transmission, cloud storage and remote data visualization.



Figure 3.13 – ThingSpeak Cloud Platform

3.7 WEEK 3 LEARNING OUTCOME

The third week provided practical knowledge of microcontrollers, sensors, actuators and IoT communication technologies. The working principles and applications of Arduino Uno and ESP8266 were understood.

Practical knowledge was gained in sensor interfacing, actuator control, OLED display interfacing and wireless communication. The working of different sensors such as DHT11, soil moisture, water level, rain and LDR sensors was studied.

The training also provided an understanding of Wi-Fi communication using ESP8266 and cloud-based data transmission using ThingSpeak. Overall, the week improved practical knowledge of embedded systems, sensor-based data acquisition and IoT communication.

WEEK 4

SMART GREENHOUSE MONITORING AND AUTOMATION SYSTEM

4.1 PROJECT OBJECTIVE

The Smart Greenhouse Monitoring and Automation System is an embedded-based system designed to monitor and manage important environmental conditions inside a greenhouse. The system uses various sensors to measure temperature, humidity, soil moisture, water level, rain, and light intensity. An Arduino microcontroller collects and processes the sensor readings and displays the real-time information on an OLED display.

The system performs automatic actions based on predefined threshold values to maintain suitable conditions for plant growth. The water pump is automatically controlled according to soil moisture levels, helping to provide water when the soil becomes dry. LED indicators and a buzzer are used to provide visual and audible alerts when abnormal conditions such as low water level, high temperature, low humidity, excessive light intensity, or rain are detected.

The sensor data is also transmitted through serial communication to an ESP8266 Wi-Fi module, which uploads the collected data to the ThingSpeak IoT cloud platform. This enables remote monitoring of greenhouse conditions through the internet. The major objectives of the project are to continuously monitor greenhouse environmental parameters, automate irrigation, provide alerts, display real-time sensor information, and enable cloud-based remote monitoring.

4.2 SYSTEM BLOCK DIAGRAM

The system block diagram represents the overall architecture of the **Smart Greenhouse Monitoring and Automation System**. Multiple sensors, including the **DHT11, soil moisture sensor, water level sensor, LDR, and rain sensor**, are connected to the **Arduino UNO**, which acts as the main controller. The Arduino collects the sensor data, processes the information, and makes decisions based on predefined threshold values.

The processed information is displayed in real time on the **OLED display**. Based on the soil moisture level, the Arduino controls the **water pump through a relay module** for automatic irrigation. **LED indicators and a buzzer** provide visual and audible alerts when abnormal environmental conditions are detected.

For remote monitoring, the sensor data is transmitted to the **ESP8266 Wi-Fi module** through communication between the Arduino and ESP8266. The ESP8266 connects to the internet and sends the collected data to the **ThingSpeak IoT cloud platform**, where the greenhouse parameters can be monitored remotely. The entire system is powered by a **5V DC power supply**, enabling continuous monitoring and automation of greenhouse conditions.

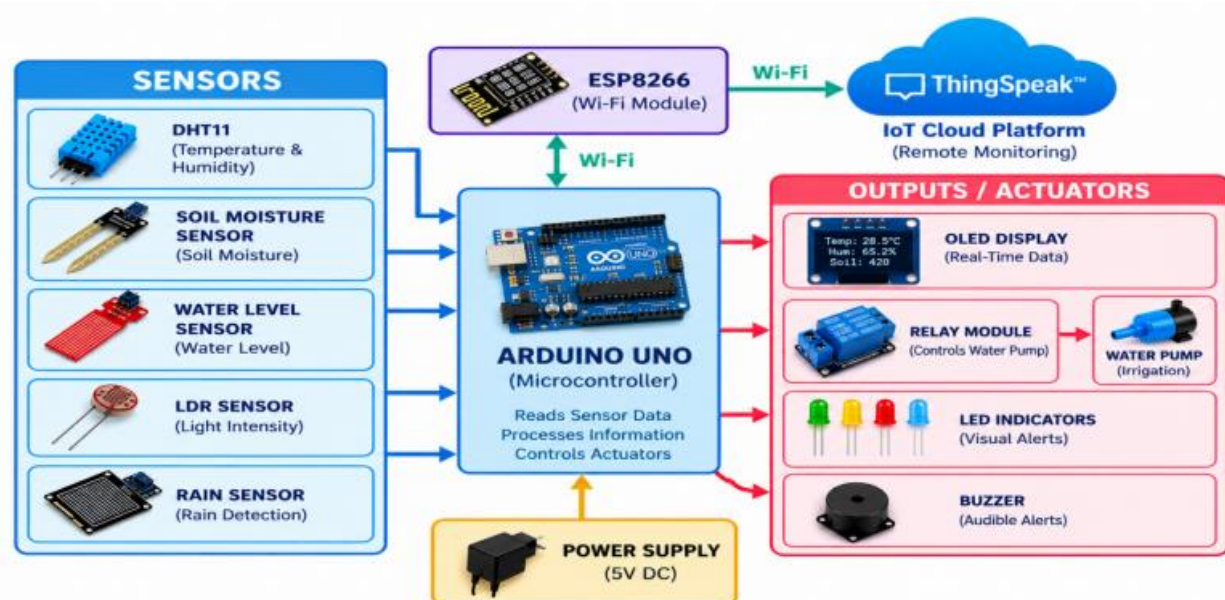


Figure 4.1 – System Block Diagram

4.3 HARDWARE COMPONENTS

COMPONENT	PURPOSE
DHT11 Sensor	Temperature and humidity measurement
Soil Moisture Sensor	Measures the moisture level of the soil
Water Level Sensor	Detects the available water level
LDR Sensor	Measures ambient light intensity
Rain Sensor	Detects the presence of rain
Water Pump	Provides automatic irrigation to plants
Relay Module	Switches the water pump ON/OFF under Arduino control
OLED Display (128×64)	Displays live sensor values
LEDs	Indicate abnormal environmental conditions
Buzzer	Provides an audible warning alert

Table 4.1 – Hardware Components

The **Smart Greenhouse Monitoring and Automation System** consists of several sensors, control devices, and output components that work together to monitor and manage greenhouse conditions. The **DHT11 sensor** measures temperature and humidity, while the **soil moisture sensor** monitors the moisture content of the soil. A **water level sensor** checks the availability of water in the storage tank, and the **LDR sensor** measures the surrounding light intensity. The **rain sensor** detects rainfall conditions to provide appropriate alerts.

The **Arduino UNO** acts as the main controller and processes the data received from all the sensors. Based on the sensor readings, it controls the **water pump through a relay module** to provide automatic irrigation when required. The **OLED display (128×64)** presents the sensor readings in real time, allowing the user to monitor the greenhouse locally.

For safety and notification, **LED indicators and a buzzer** are used to indicate abnormal environmental conditions. The **ESP8266 Wi-Fi module** provides wireless connectivity and transmits the sensor data to the **ThingSpeak IoT cloud platform** for remote monitoring. Together, these components enable continuous monitoring, automatic irrigation, environmental alerts, and IoT-based supervision of the greenhouse.

4.4 SOFTWARE USED

4.4.1 Arduino IDE

The **Arduino IDE** is used to write, compile, debug, and upload the embedded C/C++ program to the Arduino UNO. It is also used to interface and control the sensors, OLED display, relay module, water pump, LEDs, and buzzer in the Smart Greenhouse system.

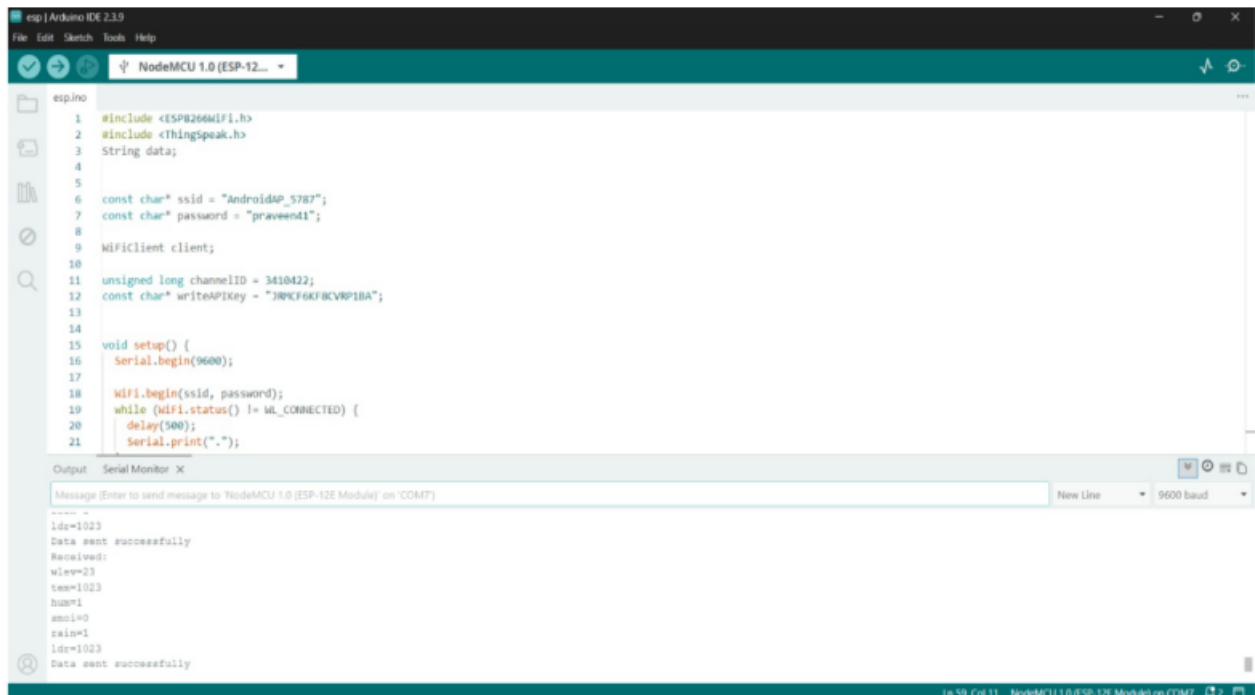


Figure 4.2 – Arduino IDE

4.4.2 ThingSpeak IoT Platform

The **ThingSpeak IoT Platform** is used for cloud-based monitoring of the greenhouse system. Sensor data received through the **ESP8266 Wi-Fi module** is uploaded to ThingSpeak, where it can be stored, displayed, and analyzed using real-time graphs. This enables remote monitoring of important greenhouse parameters such as temperature, humidity, soil moisture, water level, rain, and light intensity.

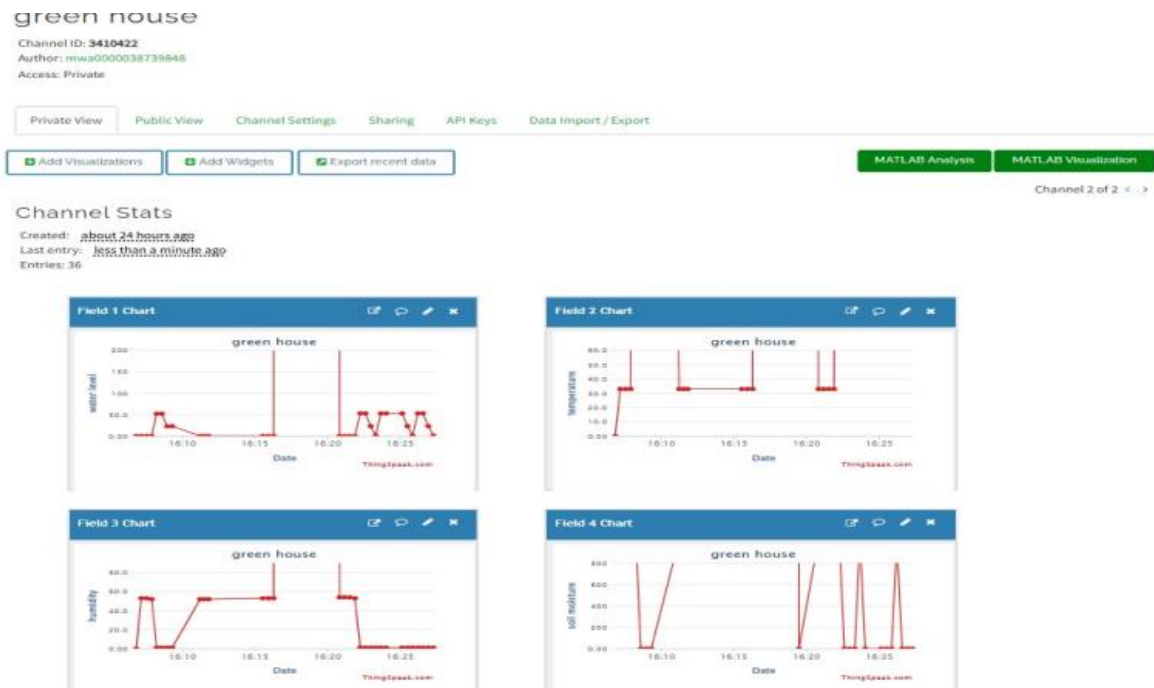


Figure 4.3 – ThingSpeak IoT Platform

4.5 WORKING PRINCIPLE

The Smart Greenhouse Monitoring and Automation System continuously monitors the environmental conditions of the greenhouse using various sensors. The **DHT11 sensor** measures temperature and humidity, while the **soil moisture sensor** monitors the moisture level of the soil. The **water level sensor** checks the amount of water available in the storage tank, and the **LDR sensor** measures light intensity. The **rain sensor** detects the presence of rainfall.

All sensor readings are collected and processed by the **Arduino microcontroller**. The processed values are displayed on the **OLED display** for real-time monitoring. Based on predefined threshold values, the Arduino automatically controls the **water pump through a relay** when the soil moisture level becomes low.

The system also uses **LED indicators and a buzzer** to provide visual and audible alerts when abnormal conditions such as low water level, high temperature, low humidity, excessive light intensity, or rain are detected.

The collected sensor data is transmitted through **serial communication to the ESP8266 Wi-Fi module**. The ESP8266 connects to the internet and uploads the sensor data to the **ThingSpeak IoT cloud platform**, allowing the greenhouse parameters to be monitored remotely. This combination of sensing, automation, and IoT connectivity reduces manual intervention and helps maintain suitable environmental conditions for plant growth.



Figure 4.4 – OLEDs

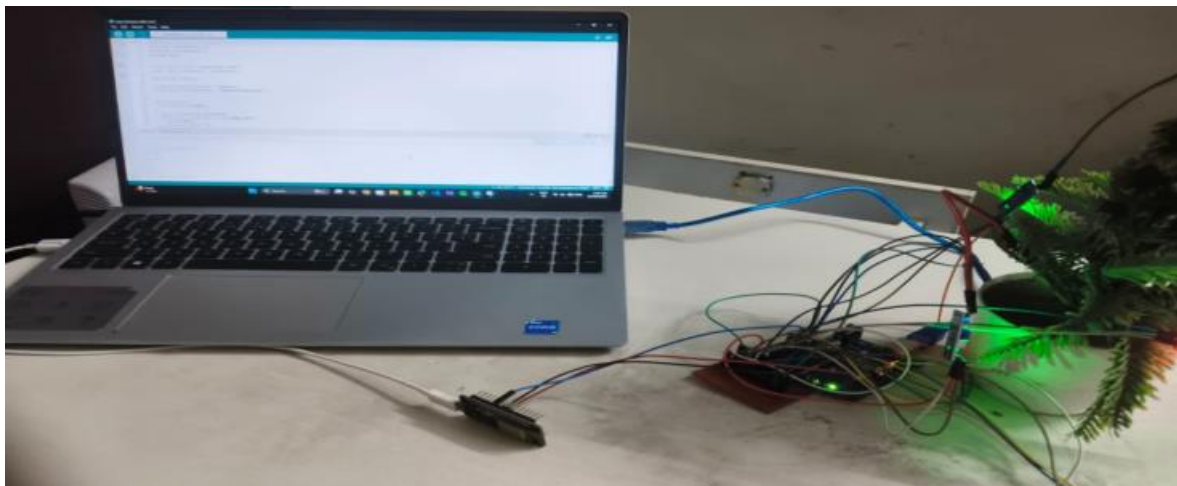


Figure 4.5 – Integrated of Components with microcontroller

4.6 ALERT SYSTEM

The alert system provides visual and audible warnings when abnormal conditions are detected in the greenhouse. LED indicators are used to indicate conditions such as low water level, high temperature, low humidity, low soil moisture, high light intensity, and rain detection. A buzzer is activated along with the indicators to immediately notify the user of critical conditions. This helps the user take timely action and maintain suitable environmental conditions for healthy plant growth.

Indicator	Condition Monitored
LED 1	Water-level condition (low water level)
LED 2	Temperature, humidity, and soil-moisture condition
LED 3	High light-intensity condition
LED 4	Rain detection
Buzzer	Activated whenever any abnormal condition above is detected

Table 4.2 – Alert System

4.7 CONCLUSION

The **Smart Greenhouse Monitoring and Automation System** successfully demonstrates the application of embedded systems and IoT technologies in smart agriculture. The system is designed to continuously monitor important environmental parameters such as **temperature, humidity, soil moisture, water level, light intensity, and rainfall** using appropriate sensors. The **Arduino UNO** acts as the central controller, collecting sensor data and processing it according to predefined conditions. The real-time sensor values are displayed on an **OLED display**, allowing the user to easily observe the greenhouse environment.

One of the major features of the system is **automatic irrigation**. The soil moisture sensor continuously checks the moisture level of the soil, and when the soil becomes sufficiently dry, the Arduino activates the **water pump through a relay module**. This helps provide water to the plants when required and reduces unnecessary water usage. The system also monitors the water level of the storage tank to help prevent problems caused by insufficient water availability.

The **alert system** further improves the reliability of the greenhouse by providing visual and audible indications when abnormal conditions are detected. LED indicators and a buzzer can alert the user to conditions such as **low water level, high temperature, low humidity, low soil moisture, high light intensity, and rainfall**. These alerts help the user take timely corrective action and maintain a suitable environment for plant growth.

The integration of the **ESP8266 Wi-Fi module and ThingSpeak IoT cloud platform** provides an additional level of monitoring. S This demonstrates how IoT connectivity can be combined with embedded control systems to create a more efficient and convenient agricultural solution.

Overall, the project provides a practical approach to **greenhouse monitoring, automatic irrigation, environmental alerting, and remote IoT-based supervision**. It reduces the need for continuous manual monitoring and supports better resource management.. Thus, the project demonstrates the potential of **Embedded Systems and IoT in developing efficient, reliable, and sustainable smart agriculture solutions**.

APPENDIX

ARDUINO IDE

ARDUINO UNO CODE

```
#include<DHT.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define wlevp A0
#define dhtp A1
#define DHTTYPE DHT11
#define smoip A2
#define ldrp A3
#define rainp 13
// #define turbidity A5
#define wpumpp 12           // water pump with relay
#define ind1  11           //water level
#define ind2  10           //temperature , moisture , humidity
// #define ind3  9           // turbidity
#define ind4  8           // ldr
#define ind5  7           //rain
#define buzz  6

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 lcd(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);
DHT dht(dhtp,DHTTYPE);

void setup()
{
  Serial.begin(9600);
  dht.begin();
  lcd.begin(SSD1306_SWITCHCAPVCC, 0x3C);
```

```

    lcd.clearDisplay();
    lcd.setTextSize(1.8);
    lcd.setTextColor(SSD1306_WHITE);
    lcd.display();
    pinMode(wlevp,INPUT);
    pinMode(dhtp,INPUT);
    pinMode(smoip,INPUT);
    pinMode(ldrp,INPUT);
    pinMode(rainp,INPUT);
    pinMode(wpumpp,OUTPUT);
    pinMode(ind1,OUTPUT);
    pinMode(ind2,OUTPUT);
    pinMode(ind4,OUTPUT);
    pinMode(ind5,OUTPUT);
    pinMode(buzz,OUTPUT);
}

```

```

void loop()
{
    int wlev,tem,hum,smoi,rain,ldr,buz;
    wlev = analogRead(wlevp);
    tem = dht.readTemperature();
    hum = dht.readHumidity();
    smoi = analogRead(smoip);
    rain = digitalRead(rainp);
    ldr = analogRead(ldrp);
    buz=0;
    lcd.display();
    lcd.clearDisplay();
    lcd.setCursor(0, 0);
    if(wlev>=70)

```

```

{
    lcd.println("waterlev :High");
}
else if (wlev>=500)
{
    lcd.println("waterlev :Medium");
}
else
{
    lcd.println("waterlev :Low");
}
lcd.setCursor(0, 25);
lcd.println("temperature ");
lcd.println(tem);
delay(1000);
lcd.display();
lcd.clearDisplay();
lcd.setCursor(0,0);
lcd.println("humidity ");
lcd.println(hum);
lcd.setCursor(0,25);
lcd.println("soilmoist");
lcd.println(smoi);
delay(1000);
lcd.display();
lcd.clearDisplay();
lcd.setCursor(0,0);

lcd.setCursor(0,25);
if(rain==0)
{
    lcd.println("Rain");
}

```

```

}
else
{
    lcd.println("not rain");
}
delay(1000);
lcd.display();
lcd.clearDisplay();
lcd.setCursor(0,0);
lcd.println("light intensity");
lcd.setCursor(0,25);
lcd.println("level:");
lcd.println(ldr);
if(smoi<= 30)
{
    digitalWrite(wpumpp, HIGH);
}
else
{
    digitalWrite(wpumpp, LOW);
}
if(wlev<=300)
{
    digitalWrite(ind1, HIGH);
    buz=1;
}
else
{
    digitalWrite(ind1,LOW);
}
if(tem>=52 || smoi<=300 || hum<=30)
{

```

```

    digitalWrite(ind2 , HIGH);
    buz=1;
}
else
{
    digitalWrite(ind2 , LOW);
}

if(ldr>=1000)
{
    digitalWrite(ind4 ,HIGH);
    buz=1;
}
else
{
    digitalWrite(ind4, LOW);
}
if(rain==0)
{
    digitalWrite(ind5, HIGH);
    buz=1;
}
else
{
    digitalWrite(ind5, LOW);
}
if(buz==1)
{
    digitalWrite(buzz, HIGH);
}
else
{

```

```
    digitalWrite(buzz , LOW);  
}  
delay(1000);  
//Serial.print("wlev,tem,hum,smoi,rain,ldr");  
Serial.print(wlev);  
Serial.print(",");  
Serial.print(tem);  
Serial.print(",");  
Serial.print(hum);  
Serial.print(",");  
Serial.print(smoi);  
Serial.print(",");  
Serial.print(rain);  
Serial.print(",");  
Serial.print(ldr);  
Serial.print("\n");  
  
}
```

ESP8266 CODE

```
#include <ESP8266WiFi.h>
#include <ThingSpeak.h>

String data;
const char* ssid = "AndroidAP_5787";
const char* password = "praveen41";

WiFiClient client;

unsigned long channelID = 3410422;
const char* writeAPIKey = "JRMCF6KF8CVRP1BA";
void setup() {
    Serial.begin(9600);

    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    Serial.println("\nWiFi Connected");
    ThingSpeak.begin(client);
}

void loop() {
    if (Serial.available()) {
        data = Serial.readStringUntil('\n');
        //Serial.print("wlev,tem,hum,smoi,rain,ldr");
        int wlev,tem,hum,smoi,rain,ldr;
```

```

    sscanf(data.c_str(), "%d,%d,%d,%d,%d,%d",&wlev, &tem, &hum, &smoi, &rain,
&ldr);

    Serial.println("Received:");
    Serial.println("wlev=" + String(wlev));
    Serial.println("tem=" + String(tem));
    Serial.println("hum=" + String(hum));
    Serial.println("smoi=" + String(smoi));
    Serial.println("rain=" + String(rain));
    Serial.println("ldr=" + String(ldr));
    ThingSpeak.setField(1, wlev);
    ThingSpeak.setField(2, tem);
    ThingSpeak.setField(3, hum);
    ThingSpeak.setField(4, smoi);
    ThingSpeak.setField(5, rain);
    ThingSpeak.setField(6, ldr);
    int status = ThingSpeak.writeFields(channelID, writeAPIKey);
    if (status == 200) {
        Serial.println("Data sent successfully");
    } else {
        Serial.print("Error code: ");
        Serial.println(status);
    }

    delay(20000);

}
}

```